

Lessons Learned Reflection

Liam Cass

Colorado State University Global

CSC400: Data Structures and Algorithms

Doctor Bingdong Li

September 13th, 2026

Lessons Learned Reflection

Nearly every piece of modern software relies on organizing and processing data efficiently, and the choice of how that data is structured can mean the difference between a program that runs instantly and one that grinds to a halt. Across the course, students in this section have learned the fundamentals of data structures and algorithms using the Java programming language, exploring the abstract data type (ADT) as a mathematical concept describing a union of data and operations and distinguishing this from a data structure, its programmatic implementation. Alongside formal notation to describe efficiency, students learned to measure and compare both algorithm and data structure implementations. This paper will summarize the lessons learned throughout this course by describing abstract data types, data structures, and algorithm efficiency.

Since the beginning of the course, students have been introduced to abstract data types and data structures. An abstract data type (ADT) is a mathematical entity that describes a set of data and the operations that can be performed on it. It may be independent of specific implementation. A data structure, by contrast, is the programmatic implementation of an ADT that defines exactly how an ADT's data and operations are carried out. Throughout the course, students studied several such ADTs (including the bag, stack, queue, and deque) and learned to select and implement the appropriate data structure based on the needs of a given problem.

Building on this foundation, students' implementations of these data structures allowed for deeper understanding of how design decisions affect any program's performance and reliability. For example, a bag structure (an unordered collection that allows duplicates), was implemented using both an array and a linked list. When using an array, a bag risks running out of capacity, thus requiring an expensive resize. When using a linked list, a bag can grow dynamically, though the cost of additional overhead and slower memory access. Across these

implementations, students learned that no single approach is universally best, and that evaluating implementation choice is essential to writing efficient, maintainable code.

With an understanding of data structures established, students turned to Big-O notation as a tool for processing and evaluating algorithms. Big-O notation is a formal method of tracking the number of steps an algorithm takes as its input grows, reducing that growth to a simplified end behavior such as constant, logarithmic, linear, or quadratic time. This framework was then applied to several sorting algorithms. Selection sort and insertion sort were seen to perform adequately on smaller datasets, while larger datasets required new techniques, such as recursion and divide-and-conquer. Merge sort and quick sort are two such examples of divide-and-conquer algorithms, which students learned performed in $O(n \times \log_2 n)$ time, a significant improvement over the $O(n^2)$ time seen with insertion and selection sort. Through this progression, students learned that matching an algorithm's approach to the size, structure, and requirements of the data being processed can create a more effective implementation.

In summary, this course provided students with the conceptual and practical foundation needed to write more effective, efficient code by teaching them to reason deliberately about data and algorithms. Students learned to distinguish an abstract data type from its concrete data structure, allowing programmers to select an implementation based on the needs of a program. Students then applied this distinction to linear data structures such as the stack, queue, and deque, learning a structure's built-in restrictions can be leveraged to solve specific problems more naturally. Finally, students used Big-O notation to evaluate algorithms, learning to match an algorithm's approach to the size and structure of the data it processes rather than relying on a single, familiar method. Taken together, these lessons demonstrate that effective coding requires deliberately choosing the data structures and algorithms best suited to a given problem.